ComicCraft - AI Model Selection

Stage 1: Research, Selection & Prompting Architecture for Generative AI Models

Project: ComicCraft Engine **Focus:** Gemini Model Suite **Document Type:** Technical Specification

1. EXECUTIVE OVERVIEW & STRATEGY

ComicCraft utilizes a multi-model generative framework to create coherent multi-panel comics. This specification details Stage 1: research, evaluation metrics, and selection of the optimal Gemini AI models for story scripting, character dialogue, panel breakdown, and visual image generation.

Text & Narrative Generation

Requires strong context memory for character bible consistency, world-building narrative flow, structural JSON outputs, and structured panel-by-panel script generation.

Visual Scene Generation

Requires persistent character design, exact spatial framing, art style retention (Manga, Western, Webcomic), and fast rendering speeds.

2. MASTER RESEARCH PROMPT (FOR SYSTEM EXECUTION)

Use the following production-grade prompt to initiate model selection analysis within Gemini tools or automated workflows:

```
PROMPT: RESEARCH AND MODEL SELECTION FOR AI COMIC CREATOR SYSTEM You are an expert AI Architect and Generative AI Researcher specializing in Multi-Agent Visual Storytelling systems. PROJECT NAME: ComicCraft - AI Comic Story Creator GOAL: Establish a structured research methodology to evaluate and select the best Generative AI models (Text-to-Text & Text-to-Image) for generating multi-panel comic strips using Google Gemini API. EVALUATION CRITERIA: 1. Scripting Model: Context window, JSON reliability, character voice consistency, panel-by-panel breakdown accuracy. 2. Image/Vision Model: Character consistency across panels, art style retention (e.g., Manga, Superhero), speech bubble layout capability, precise camera angle control. DELIVERABLES: - Model comparison for Gemini 2.5/3 Pro vs Flash models. - Workflow Architecture: Script Agent -> Visual Prompt Agent -> Panel Rendering Agent. - Cost/Latency vs Quality Optimization Matrix.
```

3. MODEL EVALUATION & SELECTION MATRIX

Based on performance benchmarks for narrative structure and visual fidelity, the following Gemini model mapping is selected for the ComicCraft pipeline:

Pipeline Task	Selected Gemini Model	Role & Capability	Latency / Cost
Story Scripting & Character Bible	Gemini Pro	Generates full storyline, character traits, tone, visual descriptions, and JSON panel breakdowns.	Medium Latency / High Quality
Dialogue & Real-time Edits	Gemini Flash	Translates dialogues, adjusts user prompts dynamically, and fine-tunes panel instructions.	Ultra-Low Latency / Cost Effective
Visual Panel Generation	Gemini Image Generation	Renders final visual panels based on detailed camera, lighting, and character appearance prompts.	High Quality / Style Consistent

4. MULTI-AGENT WORKFLOW PIPELINE

To ensure high quality and zero narrative hallucinations, ComicCraft divides the generation process into three modular pipelines:

Pipeline A: Narrative & Scripting Agent (Gemini Pro)

Takes user story idea → Creates Character Bible & Story Arc → Outputs structured panel JSON with character expressions, camera angles, sound effects (SFX), and dialogues.

Pipeline B: Art & Visual Prompt Agent (Gemini Flash)

Converts script JSON into ultra-detailed image prompts containing visual style tokens, seeds, character tags, and background environmental cues.

Pipeline C: Panel Rendering & Compositing Agent

Renders comic images using Gemini Image models, applies speech bubble overlays, positions SFX text, and constructs final grid layouts.

5. SUMMARY RECOMMENDATION

For optimal performance and cost efficiency, **Gemini Pro** should be utilized for structural narrative logic, while **Gemini Flash** powers real-time user interactions and visual prompt conditioning before panel rendering.

ComicCraft - Epic 3 Architecture

Main Application Logic Implementation in routes.py (Gemini API Integration)

Epic: Epic 3 - Backend Routes **Module:** routes.py / Flask Controller **AI Stack:** Google Gemini API

1. OBJECTIVE & API ENDPOINT ARCHITECTURE

Epic 3 implementations define the primary controller layer for `ComicCraft`. The `routes.py` file manages story prompt ingestion, coordinates multi-panel generation via Google Gemini API, renders character consistency parameters, and serves generated comic assets to the client.

Endpoint	Method	Description	Gemini Model Targeted
`/api/generate-script`	POST	Parses story prompt, character specs, and returns structured panel JSON.	Gemini 2.5 Pro
`/api/generate-panel`	POST	Generates individual comic panel image with visual consistency tags.	Gemini Flash Image
`/api/comic/<id>`	GET	Retrieves complete comic story layout, panel image URLs, and text overlays.	Database / Local Cache

2. CORE CONTROLLER IMPLEMENTATION (ROUTES.PY) - PART 1

Below is the Python Flask route handling story generation and Gemini API payload formatting:

```
# routes.py - ComicCraft Application Core Routing from flask import Blueprint, request, jsonify from google import genai from google.genai import types import os comic_bp = Blueprint('comic', __name__) client = genai.Client(api_key=os.getenv("GEMINI_API_KEY")) @comic_bp.route('/api/generate-script', methods=['POST']) def generate_script(): data = request.get_json() prompt = data.get('prompt') style = data.get('style', 'Manga') panel_count = data.get('panels', 4) system_instruction = f"""You are ComicCraft Script Writer. Generate a {panel_count}-panel comic script. Art Style: {style}. Return strict JSON with: 'title', 'characters', and 'panels' array containing (panel_number, visual_description, dialogue, sfx)."""  response = client.models.generate_content( model='gemini-2.5-pro', contents=prompt, config=types.GenerateContentConfig( system_instruction=system_instruction, response_mime_type="application/json", temperature=0.7 ) ) return jsonify({"status": "success", "data": response.text})
```

3. PANEL IMAGE GENERATION ROUTE (ROUTES.PY) - PART 2

The image generation logic handles character consistency parameters and communicates with Gemini's visual models:

```
@comic_bp.route('/api/generate-panel', methods=['POST']) def generate_panel_image():
data = request.get_json() visual_description = data.get('visual_description')
character_sheet = data.get('character_sheet', '') art_style = data.get('style', 'Comic
Book') # Combine prompt with character traits for consistency image_prompt = f"""Comic panel
in {art_style} style. Character Details: {character_sheet}. Scene: {visual_description}.
High contrast, crisp lines, vivid panel layout.""" try: image_response =
client.models.generate_images( model='imagen-3.0-generate-002', prompt=image_prompt,
config=types.GenerateImagesConfig( number_of_images=1, aspect_ratio="1:1",
output_mime_type="image/png" ) ) image_url =
save_and_get_url(image_response.generated_images[0]) return jsonify({"status": "success",
"image_url": image_url}) except Exception as e: return jsonify({"status": "error",
"message": str(e)}), 500
```

4. ERROR HANDLING & RESPONSE VALIDATION

The controller logic includes robust exception handling to ensure structured JSON responses are returned to the client even when external model timeouts or content filtering events occur.

Error Code	Trigger Cause	Handling Logic in routes.py
`400 Bad Request`	Missing `prompt` or invalid panel count in payload	Validates JSON payload prior to triggering Gemini API requests.
`422 Unprocessable`	JSON parsing failure from Gemini Pro output	Implements automatic retry logic with fallback schema hints.
`500 Server Error`	API key quota limits or model service timeout	Logs internal error details and returns sanitized message to client.

5. KEY DEPLOYMENT VERIFICATION STEPS

1. Set environment variable: `export GEMINI_API_KEY="your_api_key"`
2. Register Blueprint in `app.py`: `app.register_blueprint(comic_bp)`
3. Test endpoints with cURL or Postman prior to frontend integration.

ComicCraft Technical Architecture & Workflow

An end-to-end technical overview of system architecture, environment configurations, and deployment verification for the ComicCraft AI platform.

---

1. Project Workflow

Introduction

ComicCraft is a web-based AI comic story creator. The workflow outlines the complete sequence from initial user input to the final panel rendering and downloadable PDF generation.

System Objectives & Key Components

Section	Description	Details
Objectives	Core Goals	• Establish a clear end-to-end user sequence.• Ensure predictable state management between frontend and backend.• Maintain progress indicators for long-running AI operations.• Test both standard user actions and edge cases.
Components	System Elements	• **User Interface Form:** Captures initial story parameters.• **Task State Tracker:** Monitors background AI execution status.• **Real-time Feedback Loop:**

Section	Description	Details
		Provides loading screens and notifications.• **Final Export Module:** Generates and downloads the PDF output.

Execution Steps

1. **Entry & Input:** User opens the web application and inputs story details, character specifics, and visual style preferences.
2. **Request Dispatch:** Form data is validated client-side and forwarded to the FastAPI backend API.
3. **Processing & Panel Generation:** Backend orchestrates AI story script creation and panel image rendering.
4. **Preview & Review:** Generated panels (images, narration, dialogue) are displayed sequentially for review.
5. **PDF Export:** User triggers the PDF download action to obtain the finished comic book.

Key Takeaways

- **Predictable Flow:** State is managed synchronously across the API boundary to prevent UI desynchronization during asynchronous image generation.
- **Modular Feedback:** Continuous progress feedback ensures user engagement during multi-step model processing.

---

2. Development Environment Setup

Epic 1: Model Selection and Architecture

Introduction

Setting up the development environment involves configuring the foundational frontend and backend architecture required for running Gemini models and managing AI workflows effectively.

Objectives

- Configure local development dependencies and framework tools.
- Implement scalable modular architecture for frontend and backend components.
- Securely handle environment variables and API credentials.
- Enforce code formatting and structural standards across the codebase.

Architecture Overview

Component	Architecture Role	Description
Frontend Module	Client UI	HTML layout structure, CSS styling, and JavaScript interaction logic.
Backend Framework	API Gateway	FastAPI service for managing API endpoints, middleware, and routing.
AI Integration Layer	Model Connector	Interfaces for Google Gemini models and structured prompt builders.
Configuration	Security & Environment	Security settings, local environment configuration files, and key stores.

Environment Setup Steps

1. **Environment Initialization:** Create and activate a Python virtual environment and install project dependencies via the package manager.
2. **Architecture & Directory Setup:** Organize frontend source files (`HTML`/`CSS`/`JS`) and backend API route handlers into distinct modular directories.
3. **API Key & Security Configuration:** Store sensitive credentials and API tokens in server-side environment variables (`.env`).
4. **Model Integration:** Connect Gemini models via official SDKs for generating dialogue, narrative structure, and image generation prompts.

Key Takeaways

- **Isolation:** Virtual environments and isolated API keys ensure secure local development.
- **Modularity:** Clear separation between prompt construction logic and backend routing facilitates easier model upgrading.

---

3. Testing and Local Deployment

Epic 5: Testing and Deployment

Introduction

Testing and verifying local deployment ensures that all components—from frontend UI controls to backend AI model connections—function reliably prior to production deployment.

System Verification Scope

[User UI Controls] ⟶ [FastAPI Middleware] ⟶ [Gemini API Layer] ⟶ [PDF Renderer]

UI/UX Validation	Route Integration	Mock/Live Testing	Output Audit

Objectives & Components

- **Objectives:**
  - Validate end-to-end pipeline functionality under normal and high-load conditions.
  - Detect and address edge cases, API timeout issues, and UI rendering bugs.
  - Verify consistent responsive rendering across different screen dimensions.
  - Ensure proper error handling and fallback messages for end users.
- **Key Components:**
  - **Automated Test Suite:** Unit and integration test cases covering core API routes.
  - **UI & Responsive Testing:** Multi-device screen resolution and layout verification.
  - **Mock Service Handlers:** Simulated API responses for testing failure modes and fallback logic.

- **Deployment Verification Script:** Validates required environment variables and service health checks.

Testing Procedure

1. **Functional UI Testing:** Validate user form inputs, field requirements, and interactive button behaviors.
2. **Local Server Launch:** Start the local FastAPI server and verify live endpoint health via health check routes.
3. **End-to-End Execution:** Run test prompts to verify complete story creation and PDF output generation loops.
4. **Error Handling Checks:** Simulate API timeouts, invalid inputs, and rendering failures to confirm friendly user error messages.

Key Takeaways

- **Resilience:** Comprehensive mock testing guarantees graceful failure states when upstream API limits are reached.
- **Coverage:** Validating cross-device rendering guarantees continuous usability across varied screen form factors.

Conclusion

A robust setup combining a defined project workflow, structured model architecture, and rigorous local deployment testing ensures a seamless development lifecycle and a high-quality end-user experience for ComicCraft.